UNIVERSIDADE DE SÃO PAULO
Escola Politécnica



PCS3635 – Laboratório Digital I

Projeto final - MemoryMaster

Professor Edson midorikawa

Bancada 02

Arthur Milani Giovanini **N° USP:** 12745147
Ygor Acacio Maria **N° USP:** 12553688

SÃO PAULO, 19/03/2023

Atualização

O objetivo desse planejamento é implementar a semana 2 e o que faltou da semana 1 no desenvolvimento do projeto final “MemoryMaster” do curso de laboratório digital

Cronograma:

- **Semana 01:** G-1, M1-1, M1-2, M1-3
- **Semana 02:** M2-1, M2-2, M2-3, M2-4, M2-5
- **Semana 03:** M3-1, M4-1, M4-2, M4-3
- **Semana 04:** G-2, G-3, G-4 e ajustes finos

Pendências da semana 1

Implementar a lógica de rodadas pseudo aleatória

Implementação semana 2

Modo de Jogo 2

Código: M2-1	☑ Funcional □ Não Funcional
Requisito: Criação de jogadores	
Descrição: Representa cada jogador separadamente	
Prioridade: ☑ Alta □ Média □ Baixa	
Estabilidade: ☑ Alta □ Média □ Baixa	
***Rationale*:** Mostra rodada atual, vidas restantes, etc	
Requisitos associados: Amostragem de vidas	

Código: M2-2	☑ Funcional □ Não Funcional
Requisito: Amostragem de vidas de cada jogador	
Descrição: Mostra quantas vidas possui e reduz seu número quando erra	
Prioridade: ☑ Alta □ Média □ Baixa	
Estabilidade: ☑ Alta □ Média □ Baixa	
***Rationale*:** -	
Requisitos associados: Esquema de derrota encaminhado	

Código: M2-3	☑ Funcional □ Não Funcional
Requisito: Amostragem de qual jogador deverá jogar	
Descrição: Indica qual jogador deve jogar	
Prioridade: ☑ Alta □ Média □ Baixa	
Estabilidade: ☑ Alta □ Média □ Baixa	
***Rationale*:** - Se o jogador errado jogar, este perde	
Requisitos associados: Nenhum	

Código: M2-4	☑ Funcional □ Não Funcional
Requisito: TimeOut padrão	
Descrição: TimeOut, se for excedido, o jogador perde	
Prioridade: ☑ Alta □ Média □ Baixa	
Estabilidade: ☑ Alta □ Média □ Baixa	
***Rationale*:** - Constante, igual ao do projeto dirigido	
Requisitos associados: Nenhum	

Código: M2-5	☑ Funcional □ Não Funcional
Requisito: Esquema de vitória ou derrota (jogo 2)	
Descrição: Esquema de vitória / derrota específico ao modo 2	
Prioridade: ☑ Alta □ Média □ Baixa	
Estabilidade: ☑ Alta □ Média □ Baixa	
***Rationale*:** - Vitória de quem não errar [pode haver 2 empate]. - Ao errar uma jogada ou timeout passar, perderá - Se não errar, vencerá	
Requisitos associados: TimeOut pronto.	

Planejamento

Introdução
Durante a segunda semana implementamos o modo de jogo 2 e a lógica de rodadas pseudo aleatórias, fazendo a criação de dois jogadores, mostrando a quantidade de vida que cada jogador possui, o jogador que deve jogar em cada rodada, time Out padrão do genius e implementação da lógica de vitória e derrota.

Alterações

Completar modo de jogo 1

Modo de jogo 2
Para o funcionamento correto do modo de jogo 2 foram realizadas diversas alterações no código em relação ao projeto da semana 1. Algumas alterações, como a criação de estados e sua devida organização lógica, adição de contadores e organização do funcionamento do display de 7 segmentos.

Criação de estados
Foram criados ao todo um total de 17 novos estados, pois foi preciso diferenciar o caminho que o circuito executa entre o modo singleplayer e multiplayer.

iniRodadaMulti: Análogo ao estado de inicialização da rodada para o modo singlePlayer, porém para acessá-lo é necessário selecionar o modo 0010 e com isso mudar o caminho que os estados da máquina fará. Zera o contador de estados

mostraPlacarJogador: Estado novo para mostrar o placar no display de 7 segmento por um tempo de 5 segundos.

jogadasMulti: Estado em que o circuito espera por uma jogada dos jogadores

erroTimeOutMulti: Estado em que o jogador demora muito para executar uma jogada ou adicionar uma nova jogada.

registraMulti: Estado análogo ao de registrar jogada do modo singleplayer.
comparacaoMulti: Estado análogo ao de comparação no modo singleplayer.
proximoMulti: Estado análogo ao do modo singleplayer que vai para a próxima jogada
ultRodadaMulti:Estado análogo ao do laboratório 6 em que vai quando se está na última jogada da rodada.
proxRodadaMulti: Estado análogo ao do laboratório 6 em que antecede o estado de escrever nova jogada.

registraNovaJogadaMulti: Estado análogo ao do laboratório 6 em que registra a nova jogada feita pelo jogador.

esperaNovaJogadaMulti: Estado análogo ao do laboratório 6 em que espera pela nova jogada que o jogador deve fazer, caso contrário vai vai para erro de timeout

escreveNovaJogadaMulti: Estado análogo ao do laboratório 6 em que o jogador executa uma nova jogada e o circuito escreve na memória.

ambosGanham: Estado em que nenhum jogador perde todas as três vidas e chegam ao final das 16 rodadas.

decrementaVida1: Estado em que o circuito retira uma das vidas do jogador que errou.

decrementaVida2: Estado em que o circuito retira uma das vidas do jogador que errou.

jogador1Ganha: Estado em que o jogador 1 ganha pelo fato de que o jogador 2 perdeu todas as suas vidas

jogador2Ganha: Estado em que o jogador 1 ganha pelo fato de que o jogador 2 perdeu todas as suas vidas.

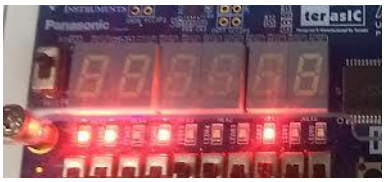
Adição de contadores

Foram adicionados dois contadores_163 no fluxo de dados, cada um para contar as vidas de cada jogador.

Lógica dos displays

mostraPlacar= 1

Jogador 1 / Vidas jogador 1 – Jogador 2 / Vidas Jogador 2 – Nulo / Jogador da rodada

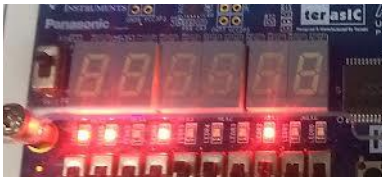


Exemplo de placar

caso: 1/3 - 2/3 - 0/1 - *jogador 1 com 3 vidas*, *jogador 2 com 3 vidas* e jogador 1 joga agora
caso: 1/3 - 2/3 - 0/2 - *jogador 1 com 3 vidas*, *jogador 2 com 3 vidas* e jogador 2 joga agora
caso: 1/2 - 2/2 - 0/1 - *jogador 1 com 2 vidas*, *jogador 2 com 3 vidas* e jogador 1 joga agora
caso: 1/1 - 2/1 - 0/2 - *jogador 1 com 1 vida*, *jogador 2 com 1 vida* e jogador 2 joga agora

mostravencedor= 1

0 / 0 – Jogador 1 ganha / Jogador 2 ganha – 0 / 0



caso: 0/0-1/0-0/0 *significa: jogador 1 ganha e jogador 2 perde*
caso: 0/0-0/1-0/0 *significa: jogador 2 ganha e jogador 1 perde*
caso: 0/0-1/1-0/0 *significa: ambos jogadores ganham*

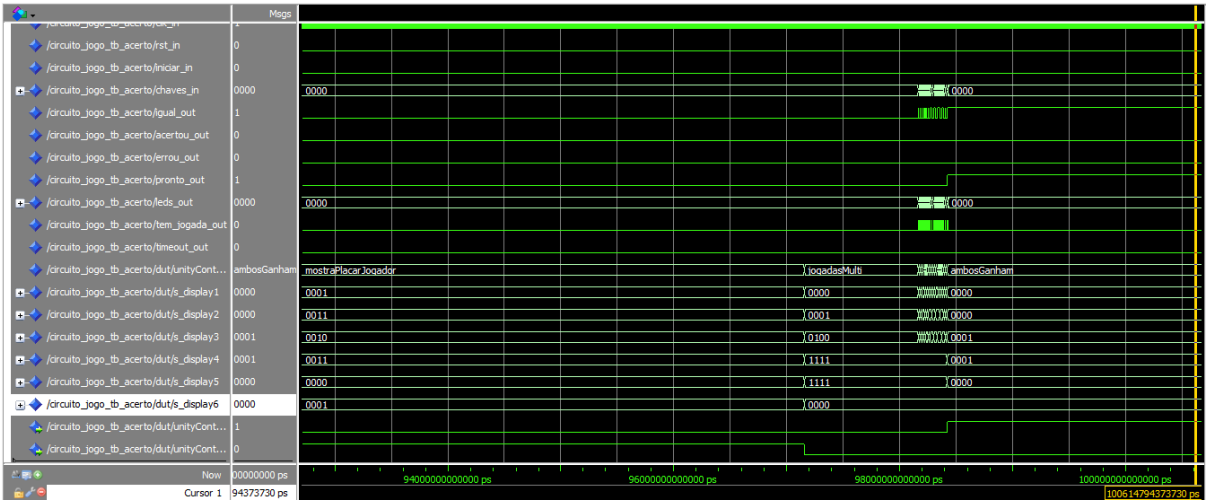
Pendências

Algumas possíveis melhorias foram identificadas, porém só foram percebidas durante a execução do projeto. Por isso, serão, caso sobrar tempo, implementadas na semana 4 em “ajustes finos”. Ademais, não foi possível implementar em casa o gerador de jogadas pseudo-aleatório.

- Alguns indicadores, como o indicador de realizar jogadas, escolher o nível possivelmente será implementado na etapa de ajustes finos
- O gerador de jogadas pseudo-aleatório será possivelmente implementado em sala, com ajuda dos monitores e professores.
- Rever a lógica de Displays
- Realização dos planos de testes

Testes modelsim

A imagem abaixo exemplifica o funcionamento do caso em que ambos os jogadores vencem (empate). Note que o circuito vai para o estado “ambosGanham” quando o sinal “pronto” é ativado. Além disso, é possível analisar o funcionamento dos sinais “mostraVencedor”, ativo apenas no final, e “mostraPlacar”, ativo apenas durante o estado “mostraPlacarJogador”.



Obs: Só foi apresentada uma imagem do ModelSim pelo fato de existirem inúmeras possibilidades no jogo: vitória com perda de vidas, perda de vidas em sequência, derrota por Time Out etc.. Em virtude disso, tais casos serão abordados no plano de testes para ser testado presencialmente na FPGA.

Plano de Testes

Caso de Erro com TimeOut de cada jogador.

- Iniciar o jogo
- Escolher o modo de jogo 2 [chaves = 0002]
- Ambos jogadores devem acertar as jogadas até a terceira (quarta) rodada
- Esperar mais que cinco segundos na terceira rodada
- Para que esteja correto, um dos jogadores deve vencer, acionando ou Display 3 (vitória do jogador 1), apresentando o valor 1, ou 4 (vitória do jogador 2), apresentando o valor 2

Caso de Erro Normal de cada jogador

- Iniciar o jogo
- Escolher o modo de jogo 2 [chaves = 0010]
- Ambos jogadores devem acertar as jogadas até a quarta (terceira) rodada
- Escolher um valor que difere da sequência correta
- Para que esteja correto, deve haver a queda da vida de um dos jogadores

Caso de acerto:

- Iniciar o jogo
- Escolher o modo de jogo 2 [chaves = 0010]
- Repetir os valores apresentados nos leds até a décima sexta rodada
- Para que esteja correto, o Led de correto e pronto deverão estar acesos

Caso de derrota jogador 1 [2]

- Iniciar o jogo
- Escolher o modo de jogo 2 [chaves = 0010]
- Repetir os valores apresentados nos leds até a rodada 5 [6];
- Fazer três erros na rodada em questão (sem ser por time out)
- Para que esteja correto, todos displays zeram, exceto o 3 ou 4. Display 4 [3] mostra 2 [1], indicando vitória do jogador 2 [1]

Funcionamento: Modo de Jogo 2

Depois de selecionar o modo de jogo “0010” o circuito redireciona o jogador para o modo multiplayer em que ambos os jogadores ficam intercalando as jogadas até que um deles perca todas as 3 vidas errando alguma jogada, caso algum deles perca por timeout o outro jogador ganha automaticamente, porém se nenhum deles zerar sua vida, ambos os jogadores ganham ao chegar na 16º rodada.

Planejamento da Aula Presencial

Segue um planejamento do que será feito na aula presencial do dia 13/03:

- Conexão dos pinos na FPGA, de acordo com as imagens do item “pinagem”
- Sintetização do circuito no Quartus
- Realização do plano de testes na FPGA
- Implementação do gerador de jogadas pseudo-aleatório em VHDL
- Testes com o gerador de jogadas pseudo-aleatório, desde o Testbech até a sintetização
- Ajustes finos das funcionalidades da semana 01 e 02

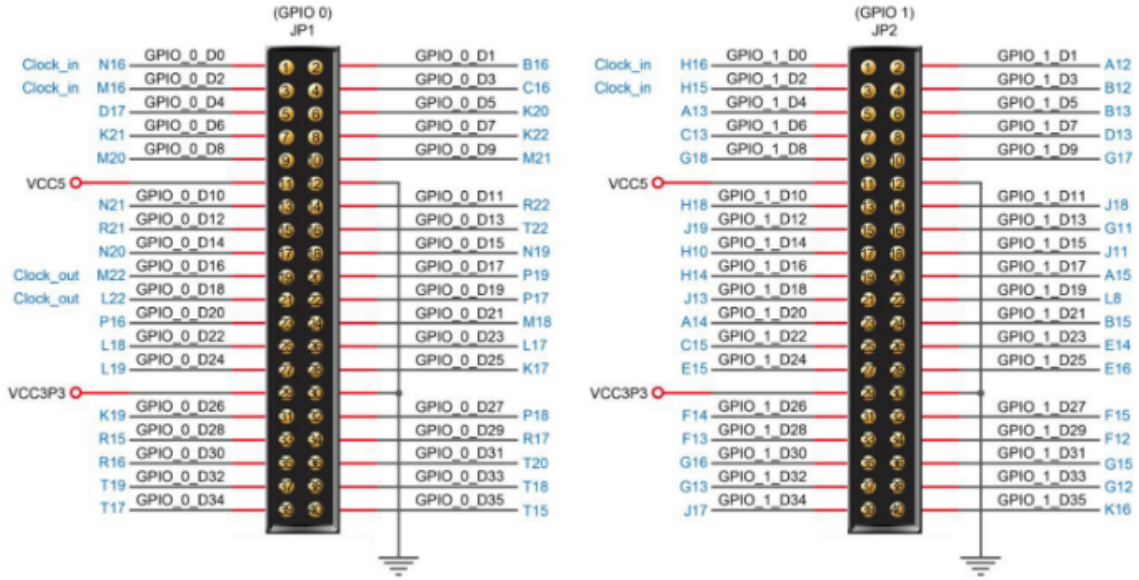
Pinagem

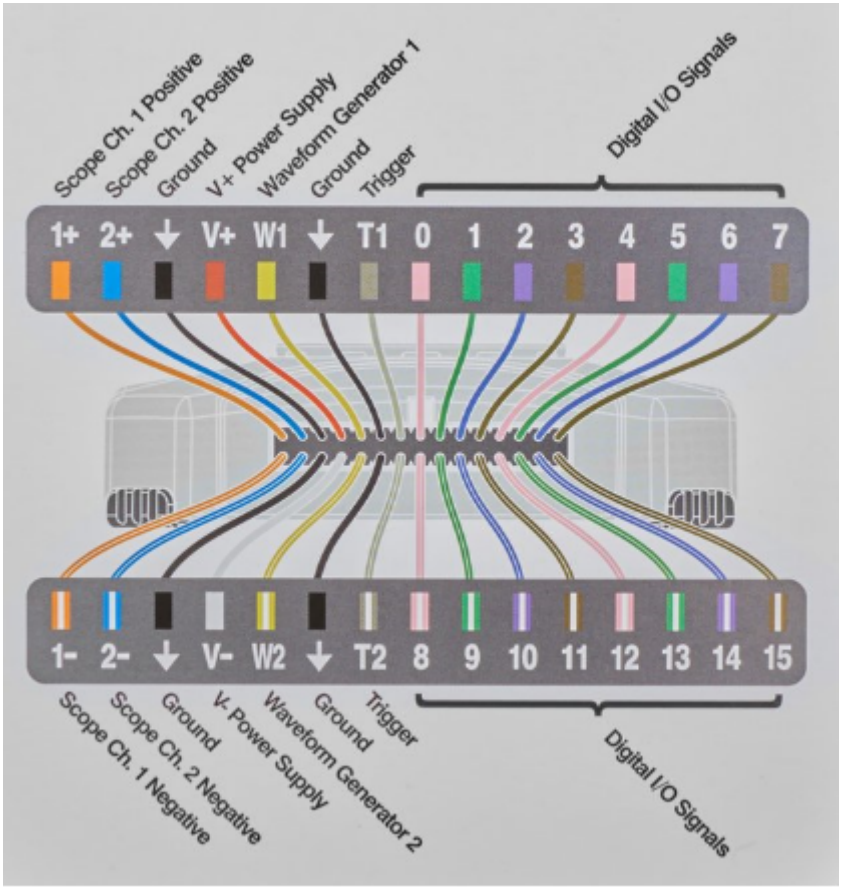
	Sinal	Pino na Placa DE0-CV	Pino no FPGA	Analog Discovery
Entradas	Clock	GPIO_0_D0	PIN_N16	Patterns – Clock – 1 KHz – DIO0
	Reset	GPIO_0_D1	PIN_B16	StaticIO – Button 0/1 – DIO1
	Iniciar	GPIO_0_D3	PIN_C16	StaticIO – Button 0/1 – DIO2
	Chaves(0)	GPIO_0_D11	PIN_R22	StaticIO – Button 0/1 – DIO4
	Chaves(1)	GPIO_0_D13	PIN_T22	StaticIO – Button 0/1 – DIO5
	Chaves(2)	GPIO_0_D15	PIN_N19	StaticIO – Button 0/1 – DIO6
	Chaves(3)	GPIO_0_D17	PIN_P19	StaticIO – Button 0/1 – DIO7
Saídas	Perdeu	GPIO_1_D11	PIN_J18	StaticIO – LED – DIO8
	Ganhou	GPIO_1_D13	PIN_G11	StaticIO – LED – DIO9
	Pronto	GPIO_1_D15	PIN_J11	StaticIO – LED – DIO10
	LEDS(0)	GPIO_1_D17	PIN_A15	StaticIO – LED – DIO12
	LEDS(1)	GPIO_1_D19	PIN_L8	StaticIO – LED – DIO13
	LEDS(2)	GPIO_1_D21	PIN_B15	StaticIO – LED – DIO14
	LEDS(3)	GPIO_1_D23	PIN_E14	StaticIO – LED – DIO15
Depuração	db_clock	led LEDR0	PIN_AA2	-
	db_tem_jogada	led LEDR1	PIN_AA1	-
	db_igual	led LEDR3	PIN_Y3	
	db_timeout	led LEDR4	PIN_N2	

	display1(0)	display HEX00	PIN_U21	
	display1(1)	display HEX01	PIN_V21	-
	display1(2)	display HEX02	PIN_W22	-
	display1(3)	display HEX03	PIN_W21	-
	display1(4)	display HEX04	PIN_Y22	
	display1(5)	display HEX05	PIN_Y21	-
	display1(6)	display HEX06	PIN_AA22	-
	display2(0)	display HEX10	PIN_AA20	-
	display2(1)	display HEX11	PIN_AB20	-
	display2(2)	display HEX12	PIN_AA19	-
	display2(3)	display HEX13	PIN_AA18	-
	display2(4)	display HEX14	PIN_AB18	-
	display2(5)	display HEX15	PIN_AA17	-
	display2(6)	display HEX16	PIN_U22	-
	display3(0)	display HEX20	PIN_Y19	-
	display3(1)	display HEX21	PIN_AB17	-
	display3(2)	display HEX22	PIN_AA10	-
	display3(3)	display HEX23	PIN_Y14	-
	display3(4)	display HEX24	PIN_V14	-
	display3(5)	display HEX25	PIN_AB22	-
	display3(6)	display HEX26	PIN_AB21	-
	display4(0)	display HEX30	PIN_Y16	
	display4(1)	display HEX31	PIN_W16	
	display4(2)	display HEX32	PIN_Y17	
	display4(3)	display HEX33	PIN_V16	
	display4(4)	display HEX34	PIN_U17	
	display4(5)	display HEX35	PIN_V18	
	display4(6)	display HEX36	PIN_V19	
	display5(0)	display HEX40	PIN_U20	-
	display5(1)	display HEX41	PIN_Y20	-
	display5(2)	display HEX42	PIN_V20	-
	display5(3)	display HEX43	PIN_U16	-

	display5(4)	display HEX44	PIN_U15	-
	display5(5)	display HEX45	PIN_Y15	-
	display5(6)	display HEX46	PIN_P9	-
	display6(0)	display HEX50	PIN_N9	
	display6(1)	display HEX51	PIN_M8	
	display6(2)	display HEX52	PIN_T14	
	display6(3)	display HEX53	PIN_P14	
	display6(4)	display HEX54	PIN_C1	
	display6(5)	display HEX55	PIN_C2	
	display6(6)	display HEX56	PIN_W19	

Configuração AnalogDiscovery





Relatório semana 02

Conexão dos pinos na FPGA, de acordo com as imagens do item “pinagem”

A conexão dos pinos da FpGA ocorreram de maneira satisfatória, além disso a configuração do anlog Discovery, também não apresentou nenhum erro .

Sintetização do circuito no Quartus

Ao sintetizar o circuito, percebeu-se que a configuração da placa FPGA na hora de gerar o QAR estava errada, para isso foi preciso configurar novamente e escolher uma nova placa FPGA nas configurações e importar a pinagem feita anteriormente.

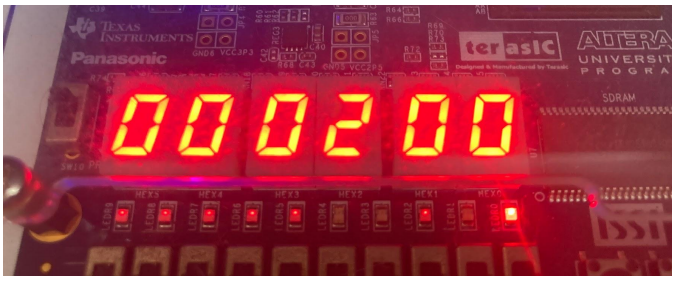
Realização do plano de testes na FPGA

O plano apresentou os seguintes bugs:

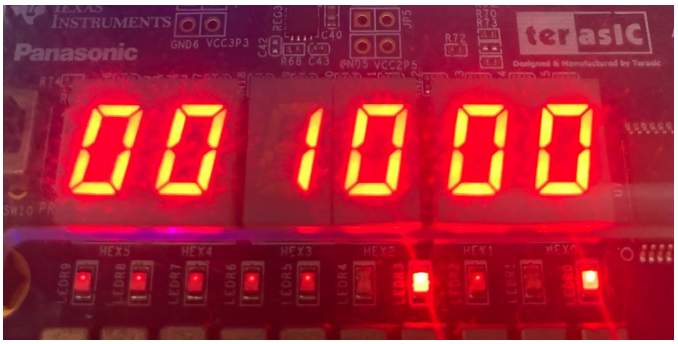
Display - organizado de forma contrária: foi necessário inverter a ordem dos sinais no fluxo de dados.
Ação de escrita de uma nova jogada não funcionava: durante o planejamento não foi notado que o sinal de escrita na memória estava desativado, para isso foi necessário repassar o sinal e escrever novamente durante o estado de esperaNovaJogada
Contagem de vida de cada jogador: o contador estava avisando que a vida dos jogadores estão chegando no máximo 4, sendo que o objetivo era 3, isso foi modificado e agora está de acordo.

Pendências

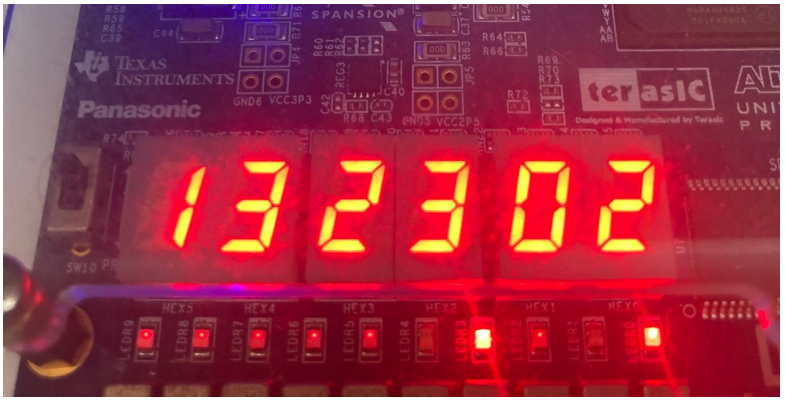
Implementar jogada pseudo-aleatória
Adicionar estado de mostraJoadaEstado no Modo de jogo 2: estado que mostra as jogadas a serem feitas por cada jogador no modo multiplayer



Mostra que o jogador 2 venceu



mostra que o jogador 1 venceu



Mostra que o jogador 1 tem 3 vidas
Mostra que o jogador 2 tem 3 vidas
mostra que o jogador 2 é o jogador atual da rodada